SLIIT — Faculty of Computing | Department of Computer Systems Engineering

IE2102 — Network Programming
FINAL EXAMINATION
Academic Year 2025/2026 — Semester 2

Duration: 2 Hours	Mode: Computer Based	Date: _____
Module Code: IE2102	Permitted Aid: One A4 Sheet (Both Sides)	Time: _____

Section	Topic	Marks
Part A — MCQ	Lectures 1, 2, 3 & SSL (Lec 9)	20 marks
Part B — Q1	I/O Multiplexing & Concurrent Servers with Signals	20 marks
Part B — Q2	DNS Programming & Data Types / Read-Write Functions	20 marks
Part B — Q3	Threads & UDP	20 marks
	TOTAL	**80 marks**

INSTRUCTIONS TO CANDIDATES:

1. This is a **Computer Based Examination**. Answer all questions on the computer system provided.
2. Part A contains **20 MCQ** questions. Select the best answer for each. Each question carries **1 mark**. No negative marking.
3. Part B contains **3 compulsory questions**. Answer ALL questions in Part B.
4. One A4 sheet (both sides) of handwritten/printed notes is permitted.
5. All questions are based on the material covered in Lectures 1–9 of IE2102.

PART A — MULTIPLE CHOICE QUESTIONS (20 Marks)

Answer ALL 20 questions. Each question carries 1 mark. Circle / select the best answer.

Q1. Which of the following best describes a **socket** in network programming? [Lec 1]

(A) A hardware component used for physical network connections
(B) An abstraction that serves as a communication endpoint identified by an IP address and port number
(C) A kernel module responsible for routing IP packets
(D) A file in the /etc directory that maps service names to port numbers

Q2. TCP is described as a **connection-oriented, full-duplex byte-stream** protocol. Which statement is **FALSE** about TCP? [Lec 1]

(A) TCP preserves message boundaries between sends and receives
(B) TCP guarantees in-order delivery of data
(C) TCP provides flow control using a sliding window mechanism
(D) TCP retransmits segments if an acknowledgment is not received within the RTT

Q3. Port numbers in the range **49152 through 65535** are classified as: [Lec 1]

(A) Well-known ports controlled by IANA
(B) Registered ports not controlled by IANA
(C) Dynamic or ephemeral (private) ports
(D) Reserved ports only accessible by the superuser

Q4. In **encapsulation**, when an application sends data using TCP, the unit of data that TCP sends to the IP layer is called a: [Lec 1]

(A) Frame
(B) IP Datagram
(C) TCP Segment
(D) Packet

Q5. Which of the following correctly describes the **sockaddr_in** structure field **sin_zero**? [Lec 2]

(A) It holds the 32-bit IPv4 address in network byte order

(B) It holds the 16-bit port number in network byte order
(C) It is unused and should always be set to zero
(D) It specifies the address family such as AF_INET

Q6. The function **htons()** converts a 16-bit value from: [Lec 2]

(A) Network byte order to host byte order
(B) Host byte order to network byte order
(C) Big-endian to little-endian always
(D) ASCII representation to binary network byte order

Q7. Which address conversion function supports **both IPv4 and IPv6** and converts a presentation (dotted-decimal) string to a network binary value? [Lec 2]

(A) inet_aton()
(B) inet_addr()
(C) inet_ntoa()
(D) inet_pton()

Q8. A **generic socket address structure** (struct sockaddr) is used in socket function calls because: [Lec 2]

(A) It is smaller and more memory-efficient than protocol-specific structures
(B) Socket functions must support multiple protocol families and need a common pointer type
(C) It automatically converts between IPv4 and IPv6 addresses
(D) It is defined by POSIX.1g and replaces all protocol-specific structures

Q9. In the TCP client-server model, which sequence of socket calls is correct for a **TCP server**? [Lec 3]

(A) socket() → connect() → read() → write() → close()
(B) socket() → bind() → listen() → accept() → read()/write() → close()
(C) socket() → listen() → bind() → accept() → connect() → close()
(D) socket() → bind() → connect() → accept() → read() → close()

Q10. The **listen()** function's **backlog** parameter specifies: [Lec 3]

(A) The maximum number of bytes that can be received in one call
(B) The maximum length of the queue of pending (incomplete) connections the kernel maintains
(C) The maximum number of simultaneous active connections the server can handle
(D) The timeout value for the accept() call in seconds

Q11. The **connect()** function called by a TCP client does NOT need to be preceded by **bind()** because: [Lec 3]

(A) The OS automatically assigns an ephemeral port number and IP address to the client socket
(B) TCP clients do not need port numbers to communicate
(C) The connect() function itself performs the binding internally using INADDR_ANY
(D) Only servers are required to call bind() in the TCP model

Q12. The **accept()** function returns: [Lec 3]

(A) The same listening socket descriptor for use in communication
(B) A new connected socket descriptor for communication with that specific client
(C) The client's process ID for signal handling
(D) Zero on success and -1 on failure, modifying the global socket

Q13. In **I/O Multiplexing** using select(), which statement is **CORRECT**? [Lec 5]

(A) select() can only monitor a single file descriptor at a time
(B) select() blocks in the actual I/O call while waiting for data
(C) select() blocks waiting for one or more of a set of descriptors to become ready, then the actual I/O is performed
(D) select() uses the SIGIO signal internally to notify the process of ready descriptors

Q14. A socket is said to be **readable** by select() under which of the following conditions? (Select the BEST answer) [Lec 5]

(A) Only when the receive buffer is completely full
(B) When the number of bytes in the receive buffer is >= the low-water mark, OR the read-half of the connection is closed (FIN received), OR it is a listening socket with completed connections
(C) Only when a full TCP segment has been received and reassembled
(D) When the write-half of the connection is closed by the peer

Q15. When a child process terminates in a concurrent server using fork(), the **SIGCHLD** signal is sent to: [Lec 4]

(A) The child process itself, causing it to clean up
(B) The kernel, which automatically removes the zombie process
(C) The parent process, which must handle it to prevent zombie processes
(D) All processes in the same process group simultaneously

Q16. The **waitpid()** function is preferred over **wait()** in a SIGCHLD signal handler for a concurrent server because: [Lec 4]

(A) waitpid() is faster and uses less memory than wait()
(B) wait() cannot be called from within a signal handler
(C) Multiple SIGCHLD signals may arrive simultaneously but only one may be delivered; waitpid() with WNOHANG in a loop handles all terminated children
(D) wait() blocks the server permanently preventing new connections

Q17. In **DNS**, a resource record of type **CNAME** is used to: [Lec 6]

(A) Map a hostname to a 32-bit IPv4 address
(B) Map a hostname to a 128-bit IPv6 address
(C) Map an IP address back to a hostname (reverse lookup)
(D) Define an alias (canonical name) for a hostname, such as www pointing to a real host

Q18. The function **gethostbyname()** is considered NOT thread-safe because: [Lec 6]

(A) It uses the UDP protocol internally which is unreliable
(B) It returns a pointer to a statically allocated hostent structure that is shared and overwritten by subsequent calls
(C) It can only resolve IPv4 addresses and not IPv6
(D) It requires root privileges to query the DNS server directly

Q19. In SSL/TLS, the **master secret K** is derived from: [Lec 9]

(A) The server's certificate and the client's password alone
(B) The pre-master secret S, the client random R_Alice, and the server random R_Bob using a pseudorandom function
(C) The session ID and the cipher suite negotiated during the handshake
(D) The Diffie-Hellman public keys exchanged during the key exchange phase

Q20. Which of the following is a key difference between **SSL v2** and **SSL v3**? [Lec 9]

(A) SSL v3 uses UDP while SSL v2 uses TCP for reliable transmission
(B) SSL v3 added a finished message with a digest of all previous handshake messages to prevent downgrade and truncation attacks present in v2
(C) SSL v2 supports authenticated encryption with additional data modes while v3 does not
(D) SSL v3 removed support for RSA key exchange to improve security

---

PART B — STRUCTURED QUESTIONS (60 Marks) | Answer ALL THREE Questions

QUESTION 01 — I/O Multiplexing & Concurrent Servers with Signals [20 Marks]

(a) I/O Models — Theory

Unix provides five I/O models for handling network I/O operations.

(i) Name and briefly describe **all five I/O models** available in Unix, clearly identifying which phase of an I/O operation each model blocks on (if any). Identify which models are classified as **synchronous** and which is truly **asynchronous** according to POSIX.1.

[6 marks]

(ii) The **I/O Multiplexing model** is considered advantageous for certain network server designs. Explain with a concrete example why a TCP echo server using a **single process with select()** is preferred over spawning a separate child process per client. Include in your answer the main **disadvantage** of I/O multiplexing compared to blocking I/O.

[4 marks]

(b) The select() Function

Consider the following partial **select()** function call context:

```
FD_ZERO(&rset;);
FD_SET(fileno(fp), &rset;);
FD_SET(sockfd, &rset;);
maxfdp1 = max(fileno(fp), sockfd) + 1;
Select(maxfdp1, &rset;, NULL, NULL, NULL);
```

(i) Explain what **maxfdp1** represents and why it is calculated as shown above.

[2 marks]

(ii) After the Select() call returns, describe the **three conditions** that can make **sockfd readable**, and explain how each condition should be handled in a TCP echo client.

[3 marks]

(c) Concurrent Server — Signals and Scenario Analysis

A network programmer writes a concurrent TCP echo server using **fork()**. The server code structure is shown below:

```
for ( ; ; ) {
    clilen = sizeof(cliaddr);
    connfd = Accept(listenfd, (SA *) &cliaddr;, &clilen;);
    if ((childpid = Fork()) == 0) {
        Close(listenfd);
        str_echo(connfd);
        exit(0);
    }
    Close(connfd);
}
```

(i) Explain the concept of **descriptor reference counts**. Why must the parent call **Close(connfd)** and the child call **Close(listenfd)** after fork()? What happens if the parent does NOT close connfd?

[3 marks]

(ii) When a client disconnects, the child process terminates. Explain what a **zombie process** is and why it is created. Write a correct **SIGCHLD signal handler** using **waitpid()** that prevents zombie processes even when **5 clients disconnect simultaneously**. Explain why **wait()** alone is insufficient in this scenario.

[5 marks] — [Marks include code quality]

QUESTION 02 — DNS Programming, Data Types & Read/Write Functions [20 Marks]

(a) Domain Name System — Concepts

(i) The DNS uses a **client-server model** with a distributed database. Briefly explain the role of the **resolver** and describe the two primary optimization techniques (Replication and Caching) that make DNS efficient despite the large number of name lookups performed globally.

[3 marks]

(ii) A DNS zone for the domain **netprog.lk** contains the following resource records:

Name	Class	Type	Value
server1	IN	A	192.168.10.5
server1	IN	MX	10 mail.netprog.lk
ftp	IN	CNAME	server1.netprog.lk
www	IN	CNAME	server1.netprog.lk
5.10.168.192.in-addr.arpa	IN	PTR	server1.netprog.lk

Explain the purpose of each record type (A, MX, CNAME, PTR). What happens when a DNS resolver queries for the IP address of **www.netprog.lk**? Trace the lookup steps.

[4 marks]

(b) DNS Functions — Programming

A programmer writes the following code to perform a hostname lookup:

```
#include <netdb.h>
struct hostent *hp;
char *hostname = "server1.netprog.lk";

hp = gethostbyname(hostname);
if (hp == NULL) {
    /* handle error */
}
```

(i) Describe the **hostent structure** returned by gethostbyname(). List and explain ALL fields of this structure with their data types.

[3 marks]

(ii) Explain why **gethostbyname()** is **NOT thread-safe**. State the thread-safe alternative function that supports both IPv4 and IPv6, and write its correct function prototype.

[2 marks]

(iii) Write a complete C code snippet that uses **gethostbyname()** to resolve the hostname "*server1.netprog.lk*" and prints the **official hostname** and **all IPv4 addresses** associated with it using **inet_ntop()**. Include proper error handling.

[4 marks] — [Marks include code quality]

(c) POSIX Data Types & Read/Write Functions

(i) Complete the following table of POSIX.1g data types used in socket programming:

Data Type	Description	Header File
`sa_family_t`		
`socklen_t`		
`in_addr_t`		
`in_port_t`		
`ssize_t`		

[2 marks]

(ii) A network programmer uses a simple **read()** loop to receive data from a TCP socket but finds that data is sometimes missing. Explain why **a single read() call may return fewer bytes than requested** from a TCP stream socket. Describe the **readn()** wrapper function — explain its purpose and how it solves this problem. Write its correct function signature.

[2 marks]

QUESTION 03 — Threads & UDP [20 Marks]

(a) POSIX Threads (Pthreads)

(i) Compare **threads** and **processes** as concurrency mechanisms for a network server. Identify **THREE specific advantages** of using threads over fork() for handling concurrent clients. List **four resources** that are shared among all threads within a process and **two resources** that are private to each individual thread.

[4 marks]

(ii) A programmer creates a concurrent TCP echo server using threads with the following code:

```
for ( ; ; ) {
    clilen = sizeof(cliaddr);
```

```
    connfd = Accept(listenfd, (SA *) &cliaddr;, &clilen;);
    Pthread_create(&tid;, NULL, &doit;, (void *) connfd);
}

void *doit(void *arg) {
    int connfd = (int) arg;
    pthread_detach(pthread_self());
    str_echo(connfd);
    Close(connfd);
    return(NULL);
}
```

(A) Identify the **race condition** in the above code. Explain clearly why it can cause incorrect behaviour.

[2 marks]

(B) Rewrite the relevant section of the server code to **correctly fix** this race condition. Explain your fix and remember to handle memory deallocation.

[3 marks] — [Marks include code quality]

(b) Mutual Exclusion and Synchronisation

Two threads in a server both execute the statement **nconn--** (decrement a shared connection counter). The C compiler translates this into three machine instructions: *Load, Decrement, Store*.

(i) Describe a specific scenario where concurrent execution of these two threads produces an **incorrect result**. Show the sequence of machine instruction execution that causes the error.

[2 marks]

(ii) Explain how a **Mutex (Mutual Exclusion lock)** solves this problem. Write a code segment using **pthread_mutex_t** that correctly protects the shared variable nconn.

[2 marks] — [Marks include code quality]

(c) UDP — Sockets and Behaviour

(i) Explain the purpose and behaviour of the **recvfrom()** and **sendto()** functions in UDP socket programming. Write the correct function prototypes for both. Explain what the **from** and **addrlen** parameters in recvfrom() contain after the function returns.

[3 marks]

(ii) A UDP client sends a request to a server but the server is **not running**. Describe exactly what happens at the client side — what does sendto() return, what error is generated, when is the error returned, and why? Explain the concept of **asynchronous errors** in UDP. What is the solution to ensure asynchronous errors are delivered to the UDP client?

[2 marks]

(iii) Explain what happens when **connect()** is called on a **UDP socket**. State the **three behavioural changes** that occur on a connected UDP socket compared to an unconnected one. Explain the **performance benefit** of using connect() when sending multiple datagrams to the same peer.

[2 marks]